Introducción a las API con FastAPI:

fundamentos, laboratorio y mini-API de catálogo

Evidencias GA1-AA1-EV01, GA1-AA1-EV02, GA1-AA2-EV03 y GA1-AA3-EV04

Integrantes

Juan Esteban Lezcano Tejada

Mariana Suárez

Emanuel Villa

Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software (ADSO)

Medellín

Instructor: Mateo Arroyave

4 de noviembre de 2026

Índice

Introducción..... 3
Objetivos..... 4
Objetivo general..... 4
Objetivos específicos..... 4
Desarrollo de la actividad..... 5
GA1-AA1-EV01. Cuestionario de reflexión inicial..... 5
GA1-AA1-EV02. Cuadro comparativo y mapa conceptual..... 7
Cuadro comparativo: Flask y FastAPI..... 7
Mapa conceptual: conceptos de una API..... 8
GA1-AA2-EV03. Práctica de laboratorio..... 9
Configuración y desarrollo de endpoints..... 9
Pruebas y evidencias desde Swagger UI..... 10
Código fuente de la práctica..... 19
GA1-AA3-EV04. Mini-API de Catálogo..... 20
Diseño e implementación del catálogo..... 20
Pruebas y evidencias de funcionamiento..... 20
Análisis de los resultados (EV04)..... 27
Código fuente de la Mini-API de Catálogo..... 28
Conclusiones..... 29
Referencias bibliográficas..... 30
Control de versiones..... 31

Introducción

Las interfaces de programación de aplicaciones (API) permiten intercambiar información entre programas mediante mecanismos definidos. En los servicios web, el protocolo HTTP establece la comunicación entre clientes y servidores, y los datos pueden transmitirse en formatos como JSON (MDN Web Docs, s. f.).

Este documento desarrolla las cuatro evidencias de la Guía 1 de FastAPI: reflexión inicial (EV01), comparación de frameworks y mapa conceptual (EV02), seis endpoints básicos (EV03) y mini-API de catálogo con cinco rutas (EV04). Ambas prácticas fueron verificadas localmente mediante Swagger UI, solicitudes HTTP y respuestas JSON (Arroyave, 2025).

Objetivos

Objetivo general

Comprender y aplicar los fundamentos de las API web mediante la reflexión conceptual, la comparación de frameworks y el desarrollo de dos prácticas REST con FastAPI, incluyendo un catálogo de productos en memoria.

Objetivos específicos

Explicar con palabras propias qué es una API y reconocer ejemplos de intercambio de datos en aplicaciones cotidianas.

Identificar las ventajas de separar frontend y backend, así como la función de GET, POST y JSON.

Comparar las características de Flask y FastAPI para el desarrollo de servicios web.

Relacionar API, HTTP, método, endpoint, JSON y código de estado mediante un mapa conceptual, y desarrollar servicios REST de laboratorio y catálogo con FastAPI.

Desarrollo de la actividad

GA1-AA1-EV01. Cuestionario de reflexión inicial

1. ¿Qué cree que es una API y para qué sirve? Explíquelo con sus propias palabras.

Considero que una API es una herramienta que permite la comunicación entre diferentes aplicaciones o sistemas informáticos. Su función principal es facilitar el intercambio de información y permitir que un programa utilice servicios de otro sin conocer su funcionamiento interno. Por ejemplo, una aplicación puede consultar información almacenada en un servidor y mostrar los resultados al usuario.

2. Mencione tres aplicaciones que use a diario y piense qué información le podrían estar pidiendo a un servidor.

Tres ejemplos de aplicaciones de uso cotidiano son WhatsApp, YouTube y una aplicación bancaria. WhatsApp consulta información relacionada con los mensajes, contactos y conversaciones; YouTube solicita videos, recomendaciones y datos de los canales; mientras que una aplicación bancaria consulta saldos, movimientos y registros de transacciones. Estas operaciones necesitan comunicación con servidores para obtener o actualizar información.

3. ¿Por qué cree que conviene separar el frontend (lo que ve el usuario) del backend (donde se procesan y guardan los datos)?

Considero que separar el frontend del backend permite organizar mejor el desarrollo de una aplicación, facilitando su mantenimiento, seguridad y actualización. El frontend se encarga de la interacción con el usuario, mientras que el backend procesa las solicitudes, aplica las reglas del sistema y administra la información. Esta separación también permite modificar una parte sin afectar necesariamente a la otra.

4. ¿Ha escuchado los términos GET, POST o JSON? ¿Qué imagina que significan?

Sí, estos términos están relacionados con el desarrollo de aplicaciones web y el intercambio de información. Entiendo que GET se utiliza para solicitar o consultar datos, mientras que POST permite enviar información al servidor, por ejemplo, al registrar un usuario. Por otra parte, JSON es un formato que organiza datos mediante claves y valores, facilitando su intercambio entre aplicaciones y servicios web.

GA1-AA1-EV02. Cuadro comparativo y mapa conceptual

Cuadro comparativo: Flask y FastAPI

La Tabla 1 compara ambos frameworks mediante ocho criterios relacionados con la construcción y documentación de API web en Python.

Tabla 1

Comparación de características de Flask y FastAPI

Criterio	Flask	FastAPI
Enfoque	Framework web ligero y flexible; permite desarrollar sitios web y API.	Framework orientado a construir API web con tipado y validación integrados.
Definición de rutas	Usa decoradores como @app.route() y @app.get().	Usa decoradores por método, como @app.get() y @app.post().
Validación de datos	Permite convertidores de ruta; validar cuerpos complejos suele requerir lógica o librerías adicionales.	Valida parámetros y datos de entrada mediante tipos de Python y modelos Pydantic.
Documentación de API	La documentación OpenAPI interactiva no viene integrada de forma predeterminada.	Genera automáticamente OpenAPI, Swagger UI (/docs) y ReDoc (/redoc).
Parámetros de ruta	Declara convertidores en la URL, por ejemplo, <int:id>.	Declara tipos en la función, por ejemplo, producto_id: int.
Asincronía	Admite funciones async, aunque su funcionamiento estándar se basa en WSGI.	Admite funciones sync y async y funciona en el entorno ASGI.
Respuestas JSON	Convierte diccionarios o listas compatibles a JSON.	Convierte automáticamente datos y modelos admitidos a JSON.
Uso en proyectos	Conveniente cuando se necesita flexibilidad para mezclar páginas web y servicios.	Conveniente para servicios REST con validación y documentación desde el inicio.

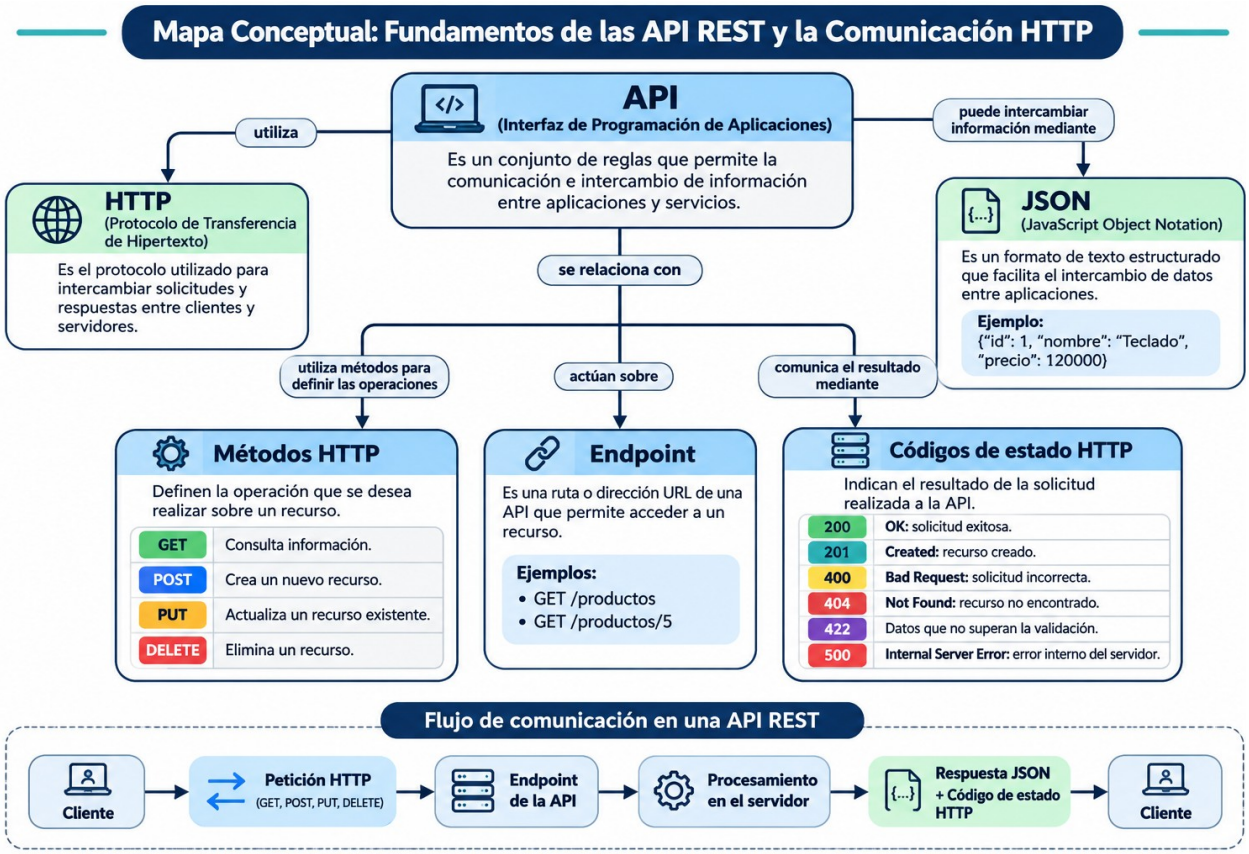
Nota. Elaboración propia a partir de la documentación oficial de Flask y FastAPI (FastAPI, s. f.; Pallets Projects, s. f.).

Mapa conceptual: conceptos de una API

La Figura 1 representa las relaciones entre las API, el protocolo HTTP, los métodos de solicitud, los endpoints, el formato JSON y los códigos de estado.

Figura 1

Relación entre API, HTTP, método, endpoint, JSON y código de estado



Nota. Elaboración propia. En una consulta GET /productos, el servidor puede responder con datos JSON y el código HTTP 200.

GA1-AA2-EV03. Práctica de laboratorio

La evidencia GA1-AA2-EV03 consistió en crear una API básica con FastAPI y comprobar el manejo de los métodos HTTP, los parámetros de ruta, los parámetros de consulta y la validación de tipos. Se trabajó de manera local en Windows con Python 3.15.0 y un entorno virtual denominado venv. La práctica siguió los pasos de la guía institucional, incluidos los tres ejercicios de apropiación (Arroyave, 2025).

Configuración y desarrollo de endpoints

Se creó el directorio mi_api y se activó el entorno virtual venv. Como la instalación de uvicorn[standard] falló al compilar PyYAML, se instalaron FastAPI y Uvicorn sin componentes opcionales. El servidor se inició con python -m uvicorn main:app --reload.

El archivo main.py contiene seis rutas GET con parámetros int, str, float y bool. FastAPI convierte las respuestas en JSON y expone la documentación interactiva en /docs (Swagger UI) y /redoc (ReDoc).

Tabla 2

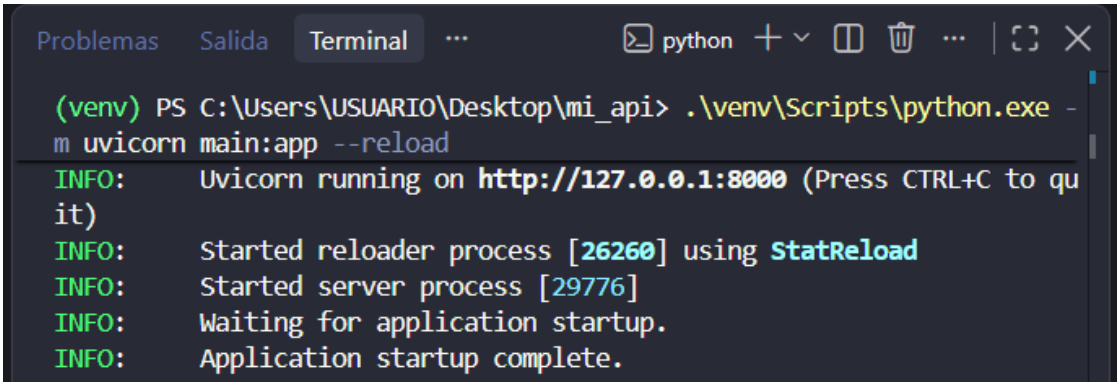
Endpoints implementados y comportamiento esperado

Ruta HTTP	Parámetros	Función de prueba
GET /	Ninguno	Devuelve el mensaje inicial.
GET /productos/{producto_id}	producto_id: int	Devuelve el ID e informa la solicitud.
GET /buscar	nombre: str; precio_max: float	Retorna los parámetros recibidos.
GET /categorias/{categoria_id}	categoria_id: int	Devuelve el ID de categoría y un mensaje.
GET /saludo/{nombre}	nombre: str	Devuelve un saludo personalizado.
GET /filtrar	categoria: str; disponible: bool	Retorna categoría y disponibilidad.

Nota. Elaboración propia con base en el código ejecutado en el laboratorio.

Figura 2

Servidor de desarrollo Uvicorn activo



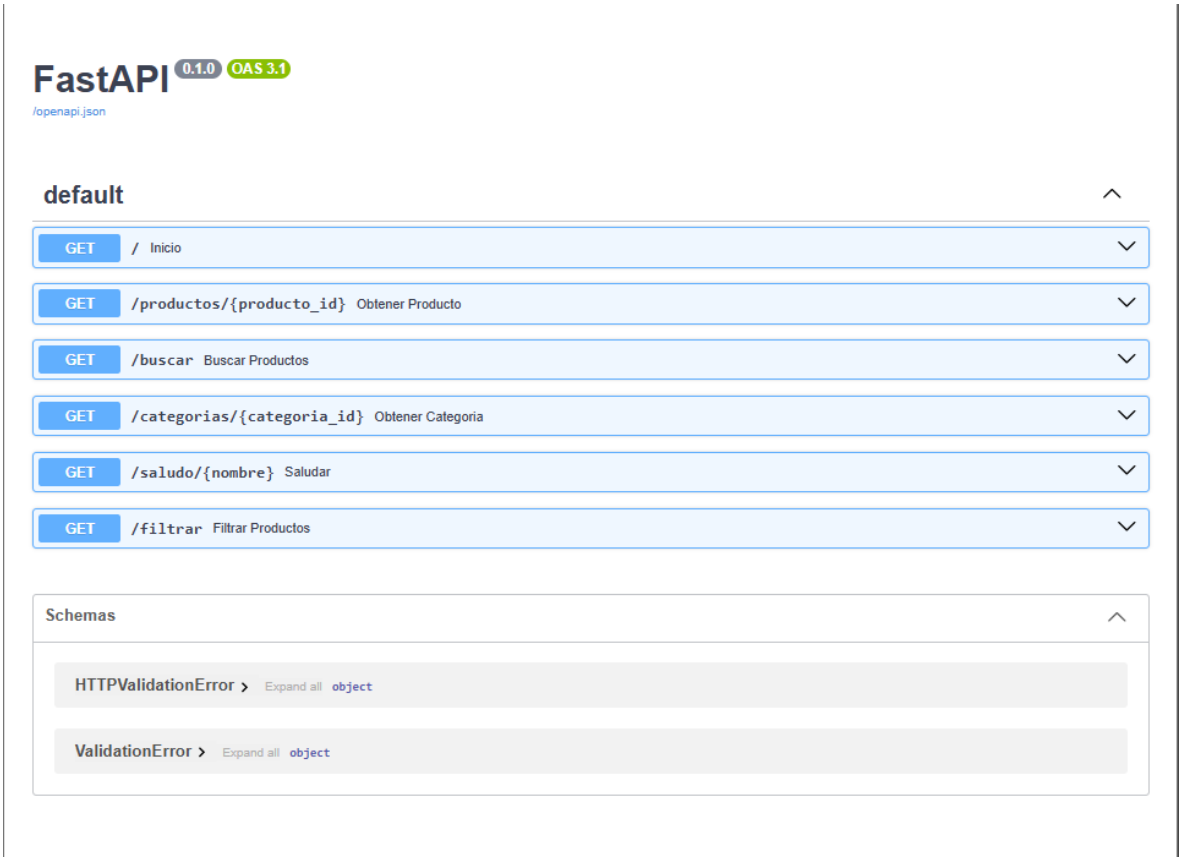
Nota. La consola registra el entorno virtual activo, el inicio de Uvicorn y la confirmación Application startup complete, que indica que la aplicación está preparada para atender solicitudes.

Pruebas y evidencias desde Swagger UI

Se verificaron los seis endpoints mediante el botón Try it out y la acción Execute de Swagger UI. En cada prueba se observó la dirección solicitada, el código de estado HTTP y el cuerpo de la respuesta. Las Figuras 3 a 9 muestran los resultados documentados. También se comprobó la respuesta de validación ante un identificador de producto no numérico, como se presenta en la Figura 10.

Figura 3

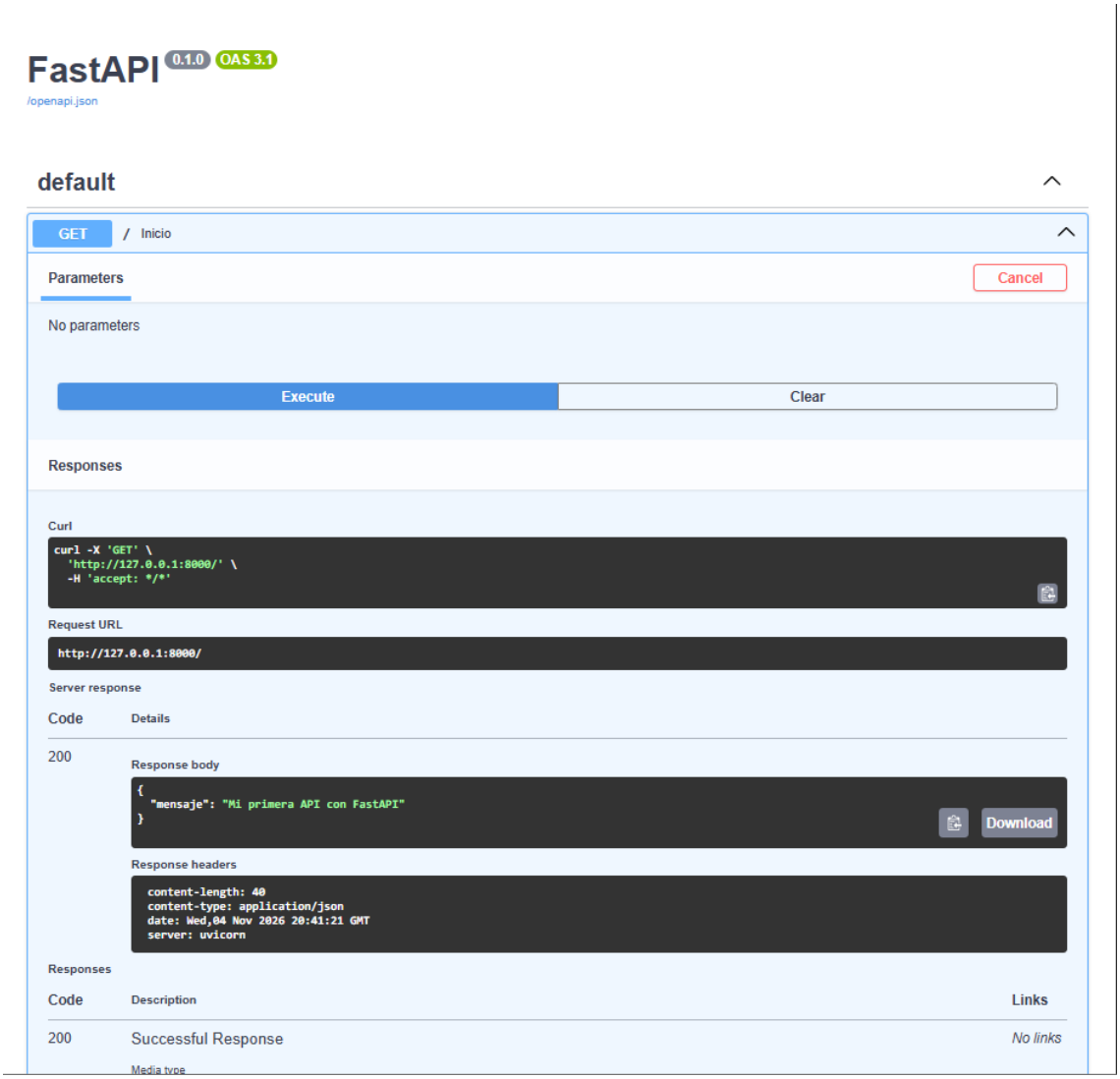
Registro de los seis endpoints en Swagger UI



Nota. Swagger UI generó la documentación de las seis rutas GET de main.py, junto con los esquemas de validación de entrada.

Figura 4

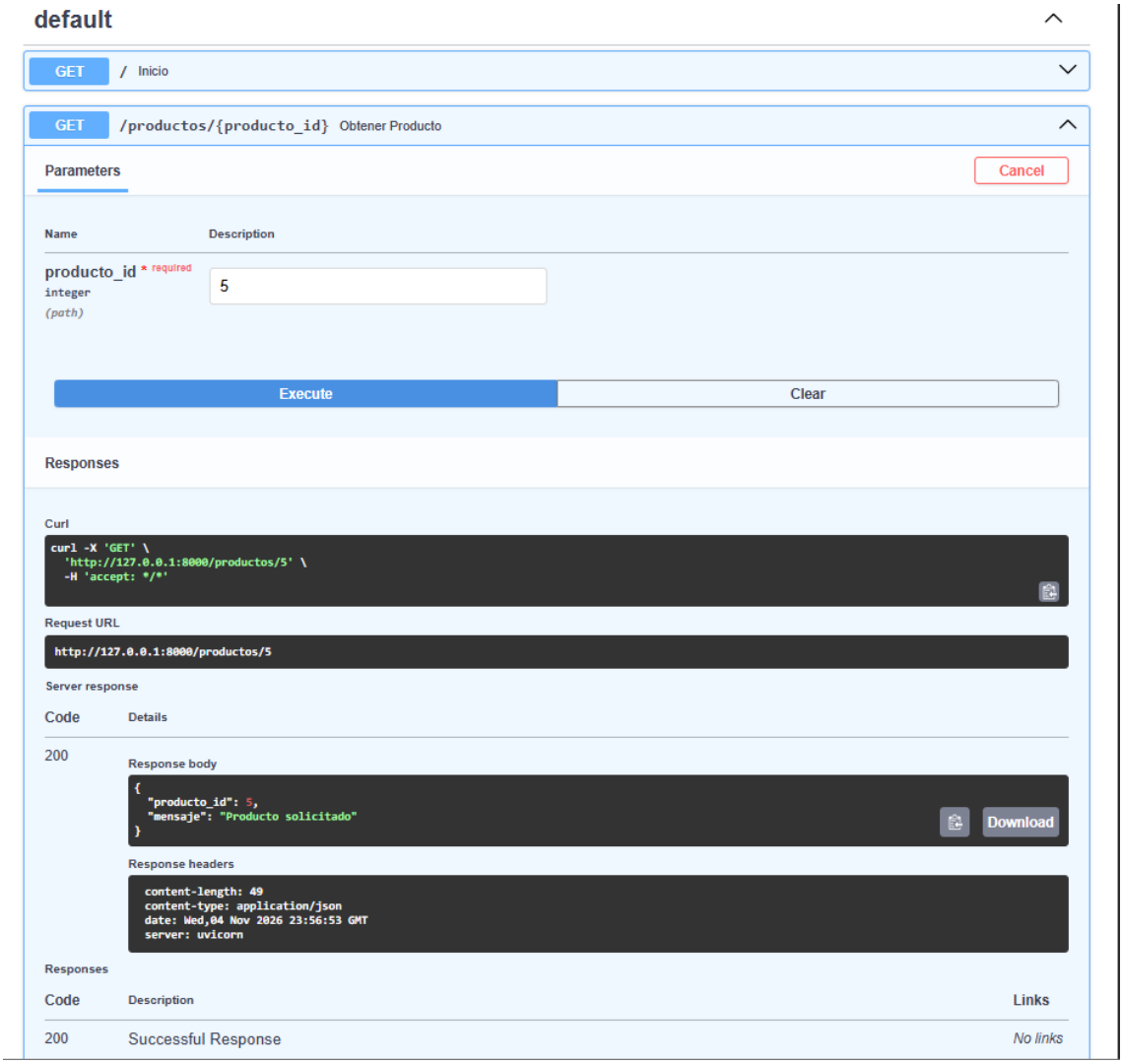
Prueba del endpoint raíz GET /



Nota. La operación GET / devolvió el código HTTP 200 y el mensaje JSON «Mi primera API con FastAPI».

Figura 5

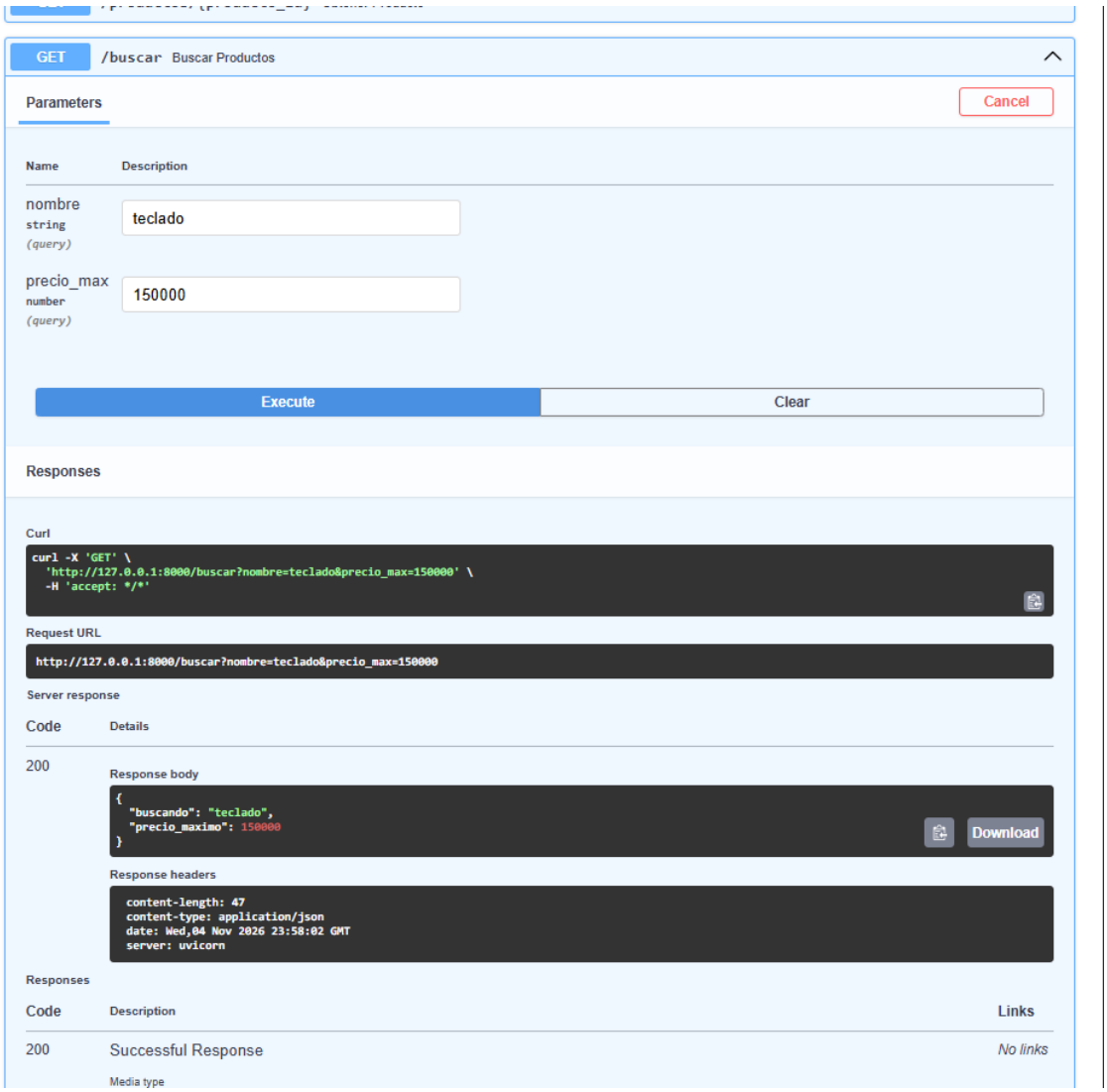
Consulta de producto mediante GET /productos/5



Nota. Se envió el identificador entero 5 y se obtuvo HTTP 200 con producto_id: 5 y el mensaje «Producto solicitado».

Figura 6

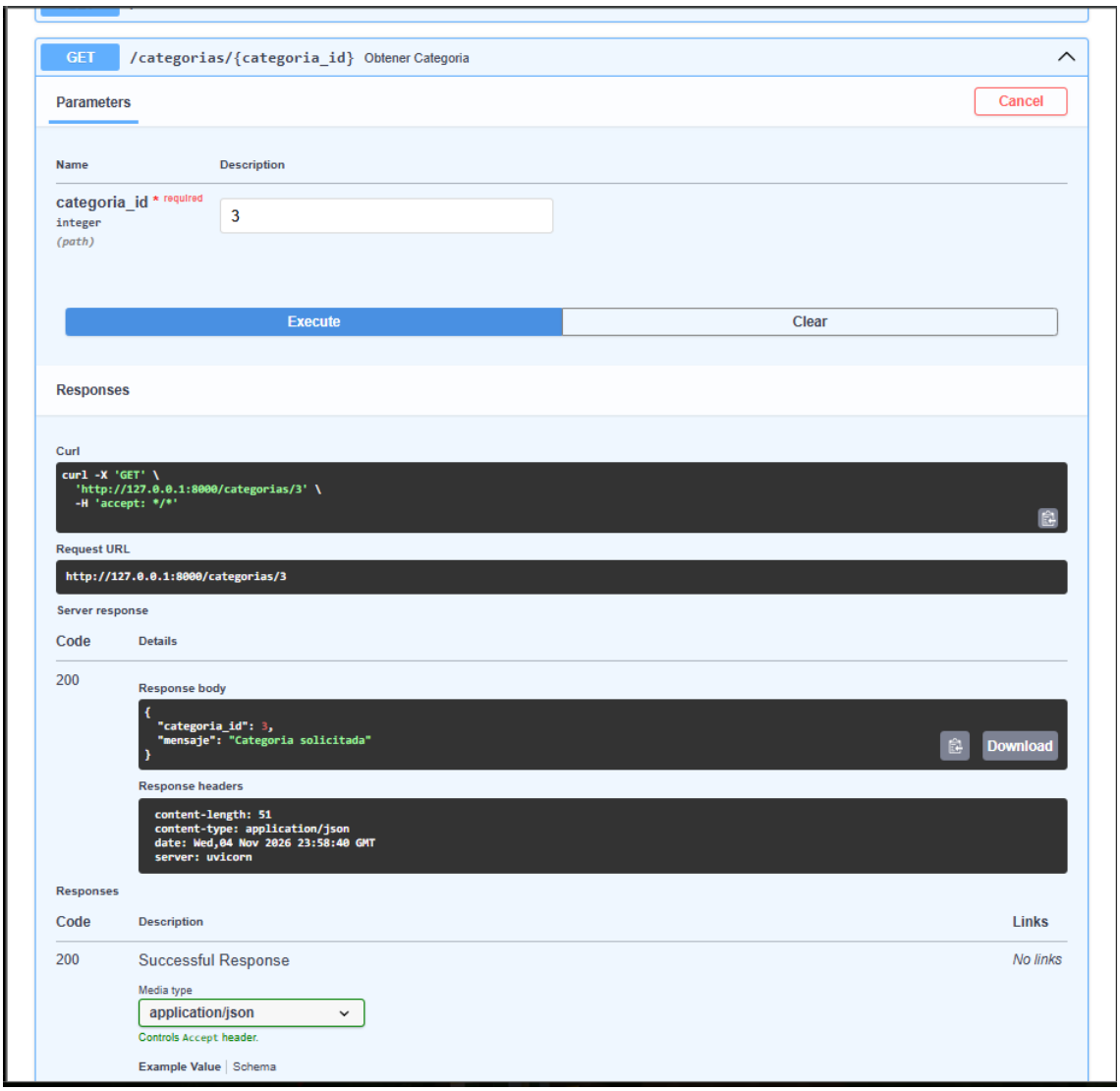
Búsqueda con parámetros de consulta



Nota. Con nombre=teclado y precio_max=150000, el endpoint GET /buscar devolvió ambos valores en formato JSON y el código HTTP 200; en esta práctica los datos se reflejan, no se filtra un catálogo real.

Figura 7

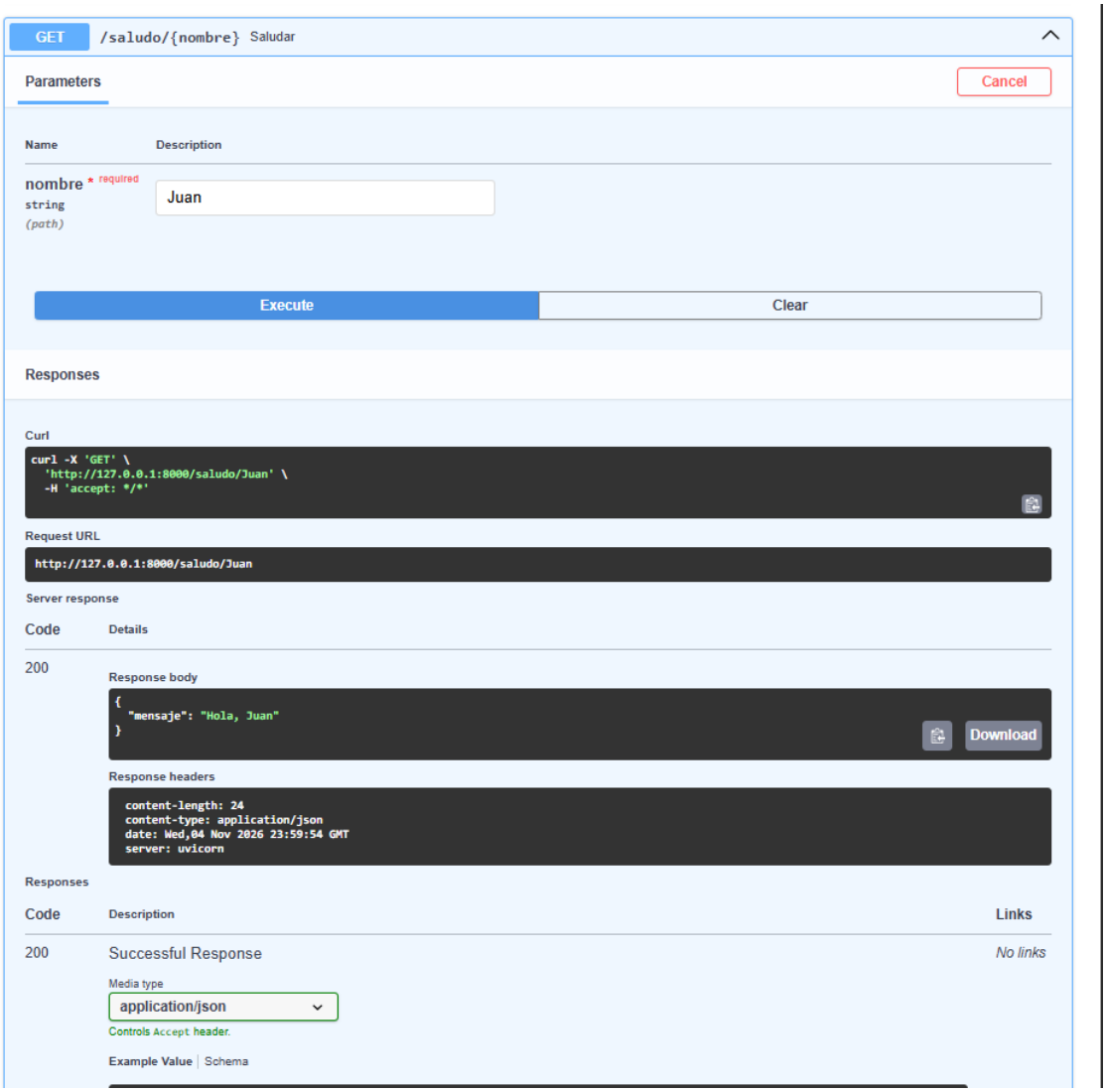
Consulta de una categoría por identificador



Nota. La ruta GET /categorias/3 devolvió categoria_id: 3 y «Categoria solicitada», con código HTTP 200.

Figura 8

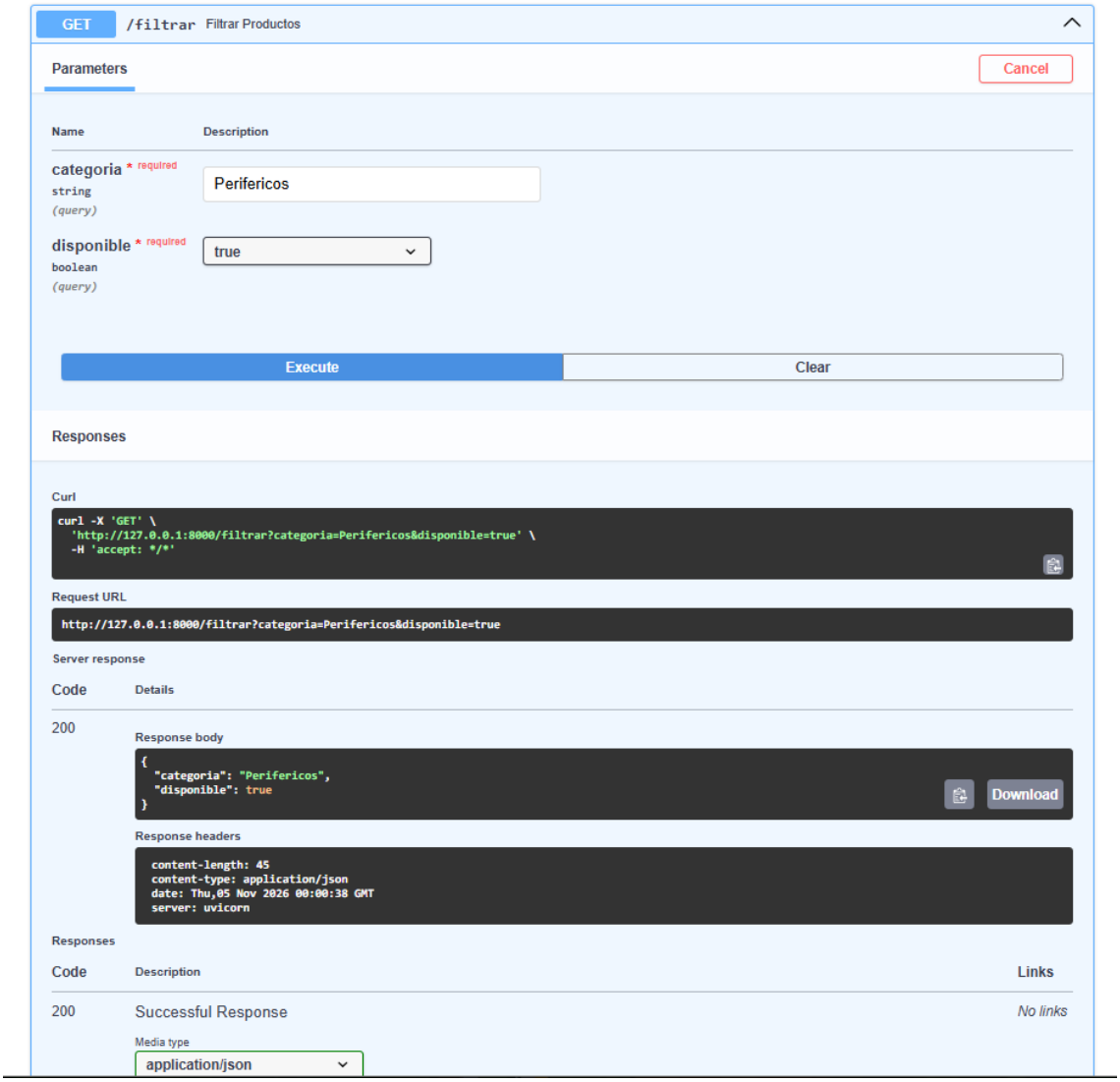
Saludo con parámetro de ruta



Nota. La ruta GET /saludo/Juan recibió un parámetro de tipo cadena y respondió con «Hola, Juan» y código HTTP 200.

Figura 9

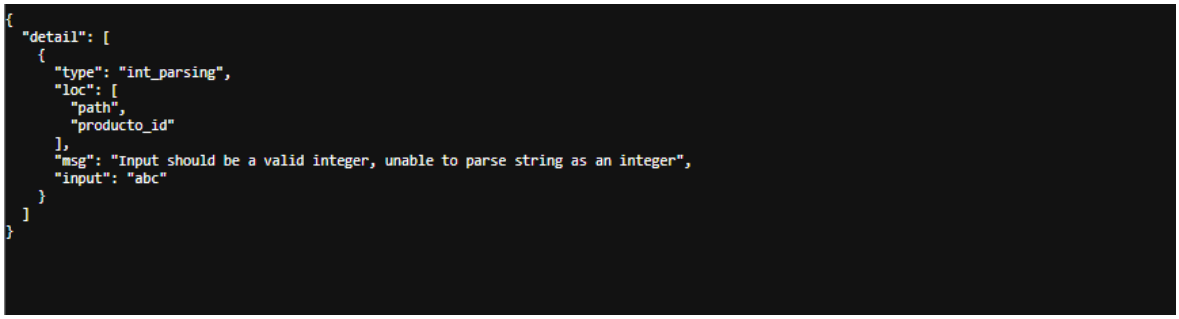
Filtro con parámetros str y bool



Nota. GET /filtrar devolvió categoria: «Pericos» y disponible: true; FastAPI interpretó el valor booleano y respondió HTTP 200.

Figura 10

Validación automática de un identificador no numérico



Nota. Al solicitar GET /productos/abc, la API informó que el valor no podía convertirse en entero. Los registros del servidor mostraron el estado HTTP 422, que evidencia la validación automática del tipo int.

Análisis de los resultados

Las seis pruebas positivas mostraron respuestas HTTP 200 y cuerpos JSON coherentes con los valores enviados. La operación /productos/{producto_id} evidenció la validación del tipo entero: un valor numérico se procesó correctamente y el texto abc produjo un error de validación HTTP 422. Por su parte, /filtrar comprobó que el valor true se interpreta como booleano. Estas pruebas permiten verificar el funcionamiento de las rutas y su documentación interactiva, sin implicar todavía persistencia de información ni conexión con una base de datos.

Código fuente de la práctica

El siguiente listado reproduce la implementación completa de main.py utilizada en esta evidencia. Se preservan los seis endpoints para que la práctica pueda reproducirse sin interferir con la Mini-API de Catálogo de la evidencia posterior.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def inicio():
    return {"mensaje": "Mi primera API con FastAPI"}

@app.get("/productos/{producto_id}")
def obtener_producto(producto_id: int):
    return {"producto_id": producto_id,
            "mensaje": "Producto solicitado"}

@app.get("/buscar")
def buscar_productos(nombre: str = "", precio_max: float = 0):
    return {"buscando": nombre, "precio_maximo": precio_max}

@app.get("/categorias/{categoria_id}")
def obtener_categoria(categoria_id: int):
    return {"categoria_id": categoria_id,
            "mensaje": "Categoria solicitada"}

@app.get("/saludo/{nombre}")
def saludar(nombre: str):
    return {"mensaje": f"Hola, {nombre}"}

@app.get("/filtrar")
def filtrar_productos(categoria: str, disponible: bool):
    return {"categoria": categoria, "disponible": disponible}
```

GA1-AA3-EV04. Mini-API de Catálogo

La evidencia GA1-AA3-EV04 consistió en desarrollar una API REST de solo lectura para un catálogo de productos tecnológicos. De acuerdo con la guía, los datos permanecen en una lista de diccionarios de Python dentro del archivo catalogo_ev04/main.py; no se utiliza una base de datos persistente. La aplicación fue ejecutada con Uvicorn y sus endpoints fueron probados mediante Swagger UI (Arroyave, 2025).

Diseño e implementación del catálogo

El catálogo contiene tres productos de ejemplo: Teclado mecanico (120000 COP), Mouse gamer (85000 COP) y Monitor 24 (650000 COP). Cada registro dispone de identificador, nombre, precio y categoría. El endpoint de consulta por identificador recorre la lista y devuelve un producto específico o una respuesta HTTP 404 cuando el identificador no existe. La búsqueda por nombre compara los textos en minúsculas, lo que permite coincidencias parciales sin distinción de mayúsculas y minúsculas.

Tabla 3
Endpoints de la Mini-API de Catálogo

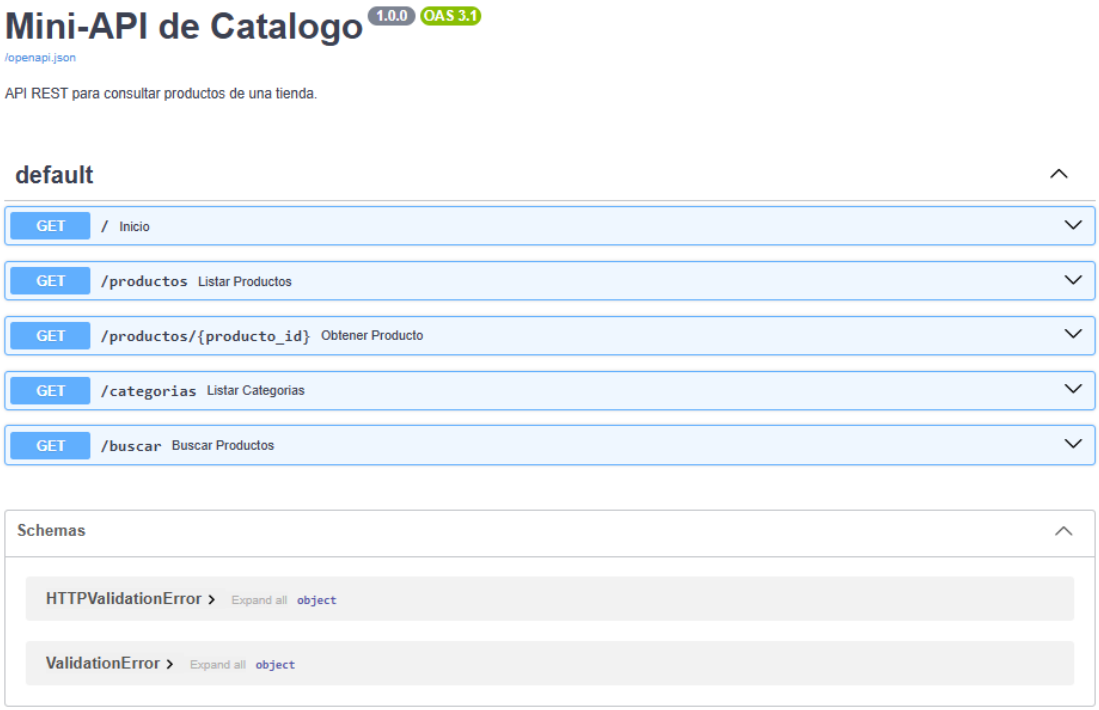
Método	Ruta	Comportamiento
GET	/	Devuelve mensaje de bienvenida
GET	/productos	Lista los tres productos
GET	/productos/{producto_id}	Consulta un producto por ID (404 si no existe)
GET	/categorias	Devuelve categorías distintas
GET	/buscar?nombre=...	Filtra los productos por nombre

Nota. Elaboración propia a partir del código de catalogo_ev04/main.py y de los requisitos funcionales de la guía.

Pruebas y evidencias de funcionamiento

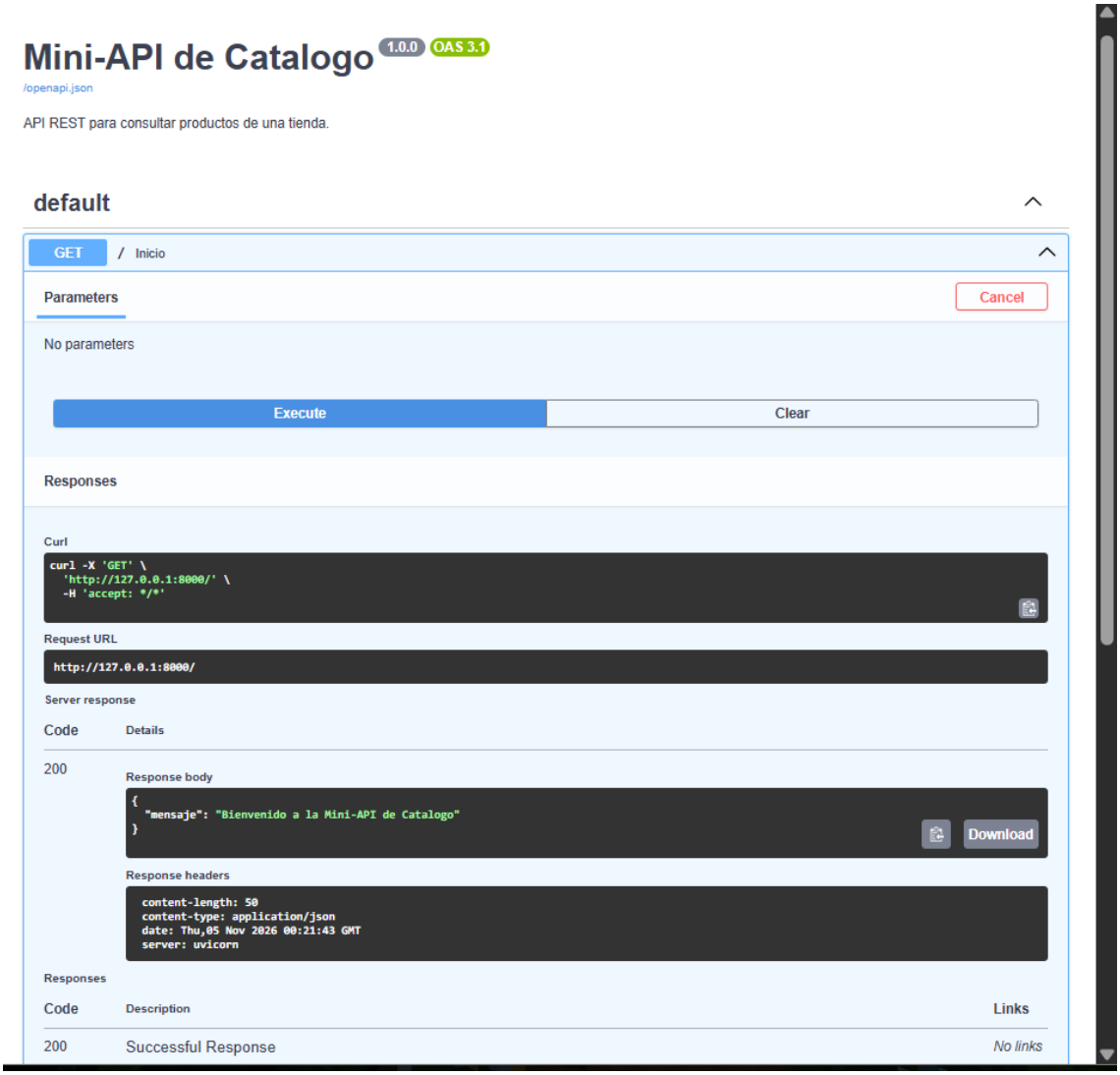
Se ejecutaron solicitudes HTTP GET desde la documentación interactiva /docs, utilizando las opciones Try it out y Execute. Se comprobaron los cinco endpoints obligatorios y un caso negativo adicional, que permitió validar el manejo de productos inexistentes. Las capturas siguientes muestran los valores enviados, el código HTTP obtenido y la respuesta JSON generada por el servidor.

Figura 11
Documentación de los cinco endpoints del catálogo



Nota. Swagger UI muestra las cinco rutas GET exigidas: inicio, lista de productos, consulta por ID, categorías y búsqueda.

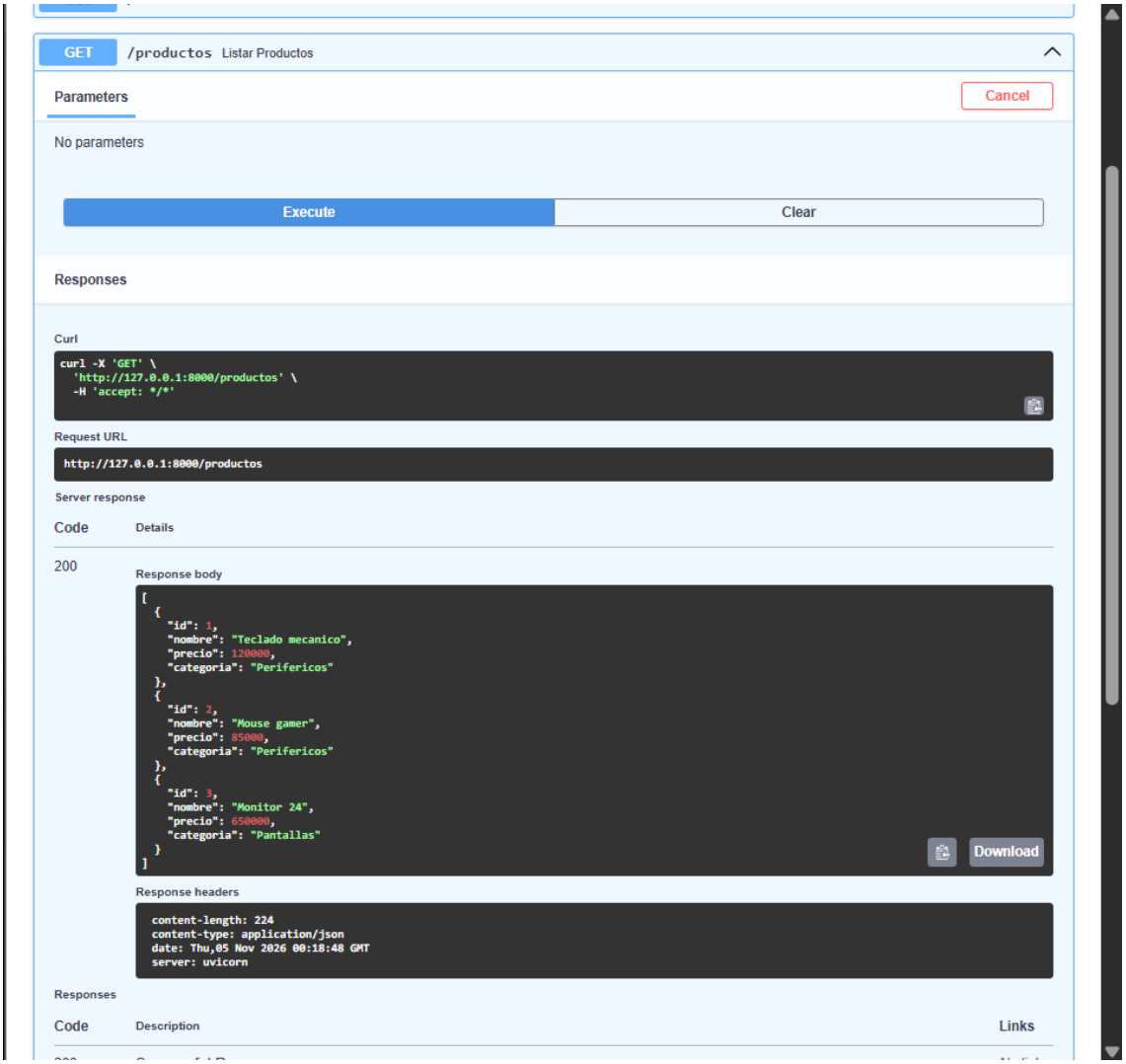
Figura 12
Mensaje de bienvenida del catálogo



Nota. La ruta GET / devolvió el mensaje «Bienvenido a la Mini-API de Catalogo» con código HTTP 200.

Figura 13

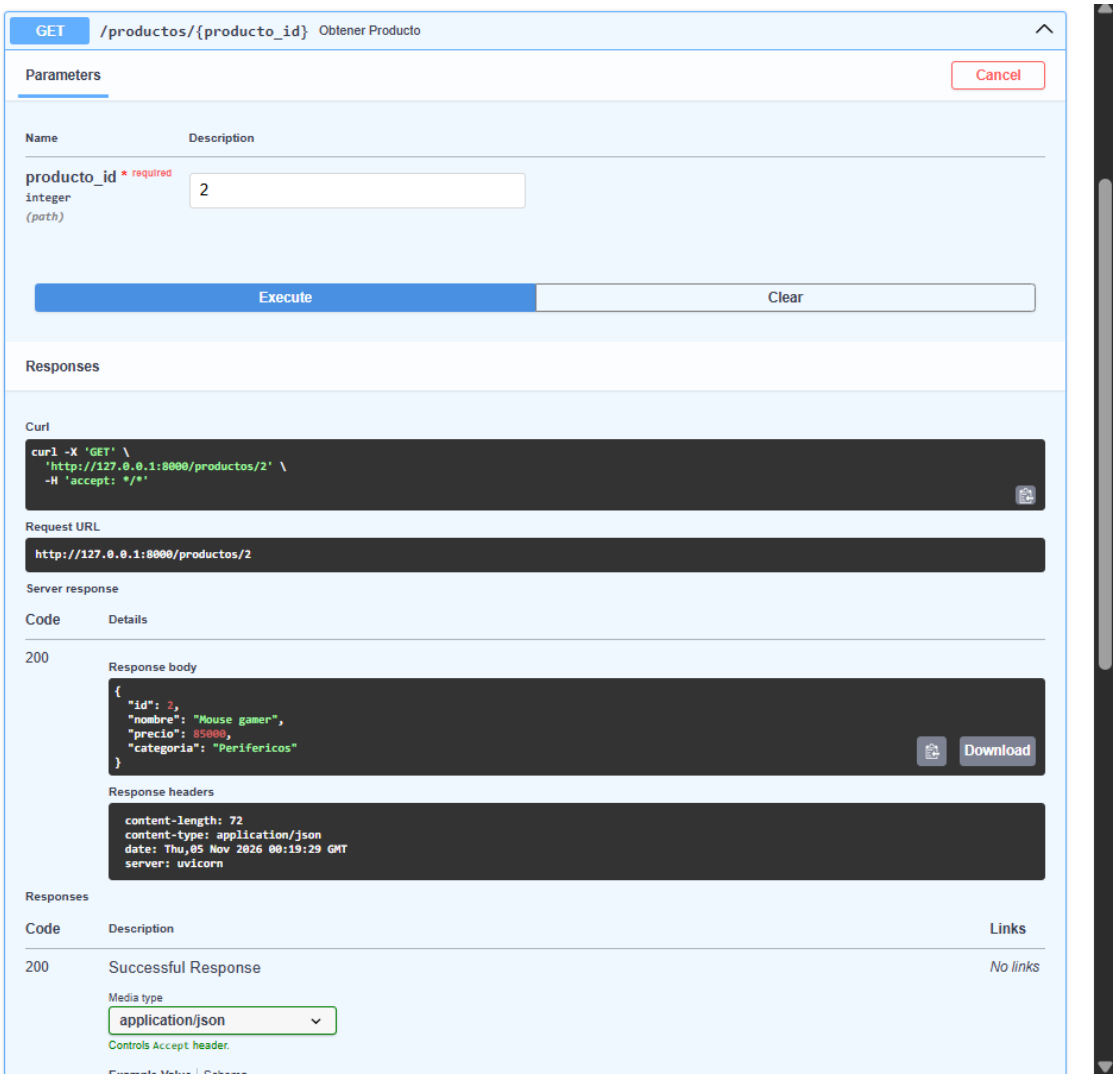
Listado de productos almacenados en memoria



Nota. La solicitud GET /productos devolvió los tres registros del catálogo (teclado, mouse y monitor), con código HTTP 200.

Figura 14

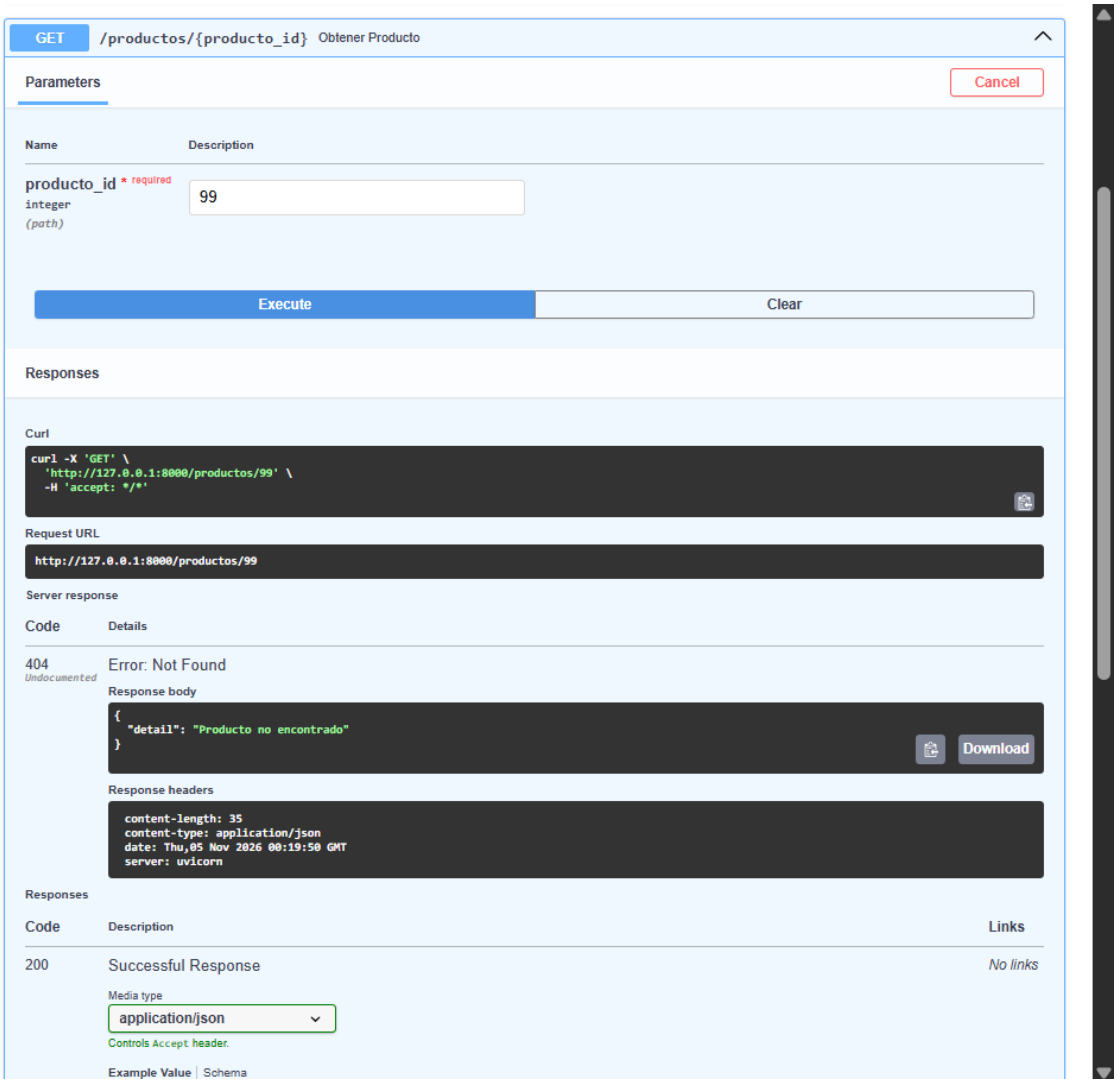
Consulta individual del producto con ID 2



Nota. La solicitud GET /productos/2 obtuvo el registro Mouse gamer, con precio 85000 y categoría Perifericos; respuesta HTTP 200.

Figura 15

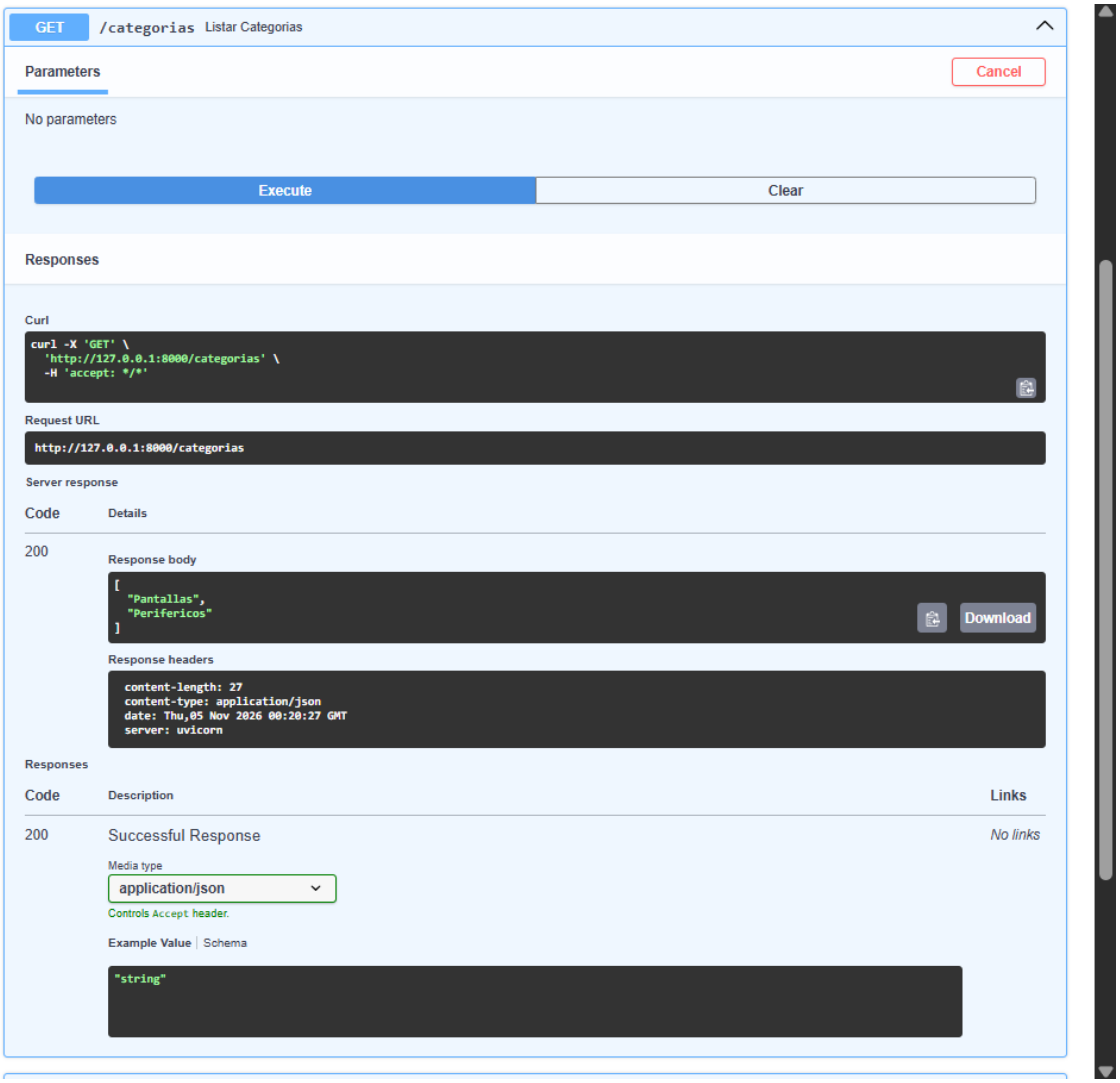
Respuesta ante un identificador inexistente



Nota. La consulta GET /productos/99 devolvió HTTP 404 y el detalle «Producto no encontrado». Se trata de una respuesta esperada para un recurso inexistente.

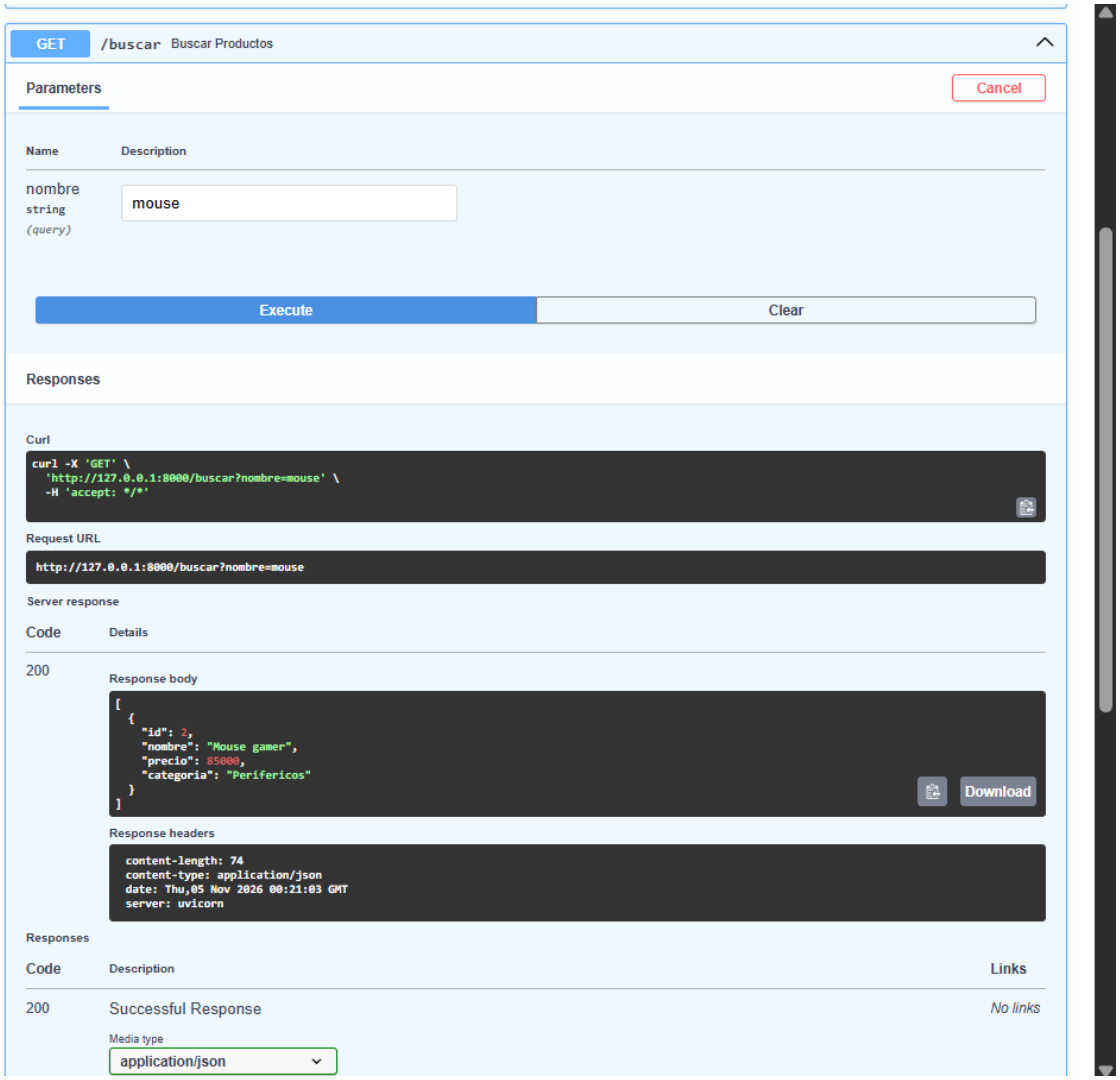
Figura 16

Obtención de categorías sin duplicados



Nota. La ruta GET /categorias devolvió dos valores distintos, «Pantallas» y «Perifericos», con código HTTP 200.

Figura 17
Búsqueda de productos por nombre



Nota. La solicitud GET /buscar?nombre=mouse devolvió únicamente el registro Mouse gamer y respondió HTTP 200.

Análisis de los resultados

Las pruebas documentadas muestran que la Mini-API cumple sus cinco funciones de solo lectura. El listado entrega todos los registros disponibles, la consulta individual localiza el elemento solicitado y el endpoint de categorías evita duplicados. La búsqueda por coincidencia parcial encuentra un producto mediante la cadena «mouse». Además, el caso con ID 99 demuestra una respuesta controlada HTTP 404 para un recurso inexistente, sin alterar los registros en memoria.

Código fuente de la Mini-API de Catálogo

El siguiente código implementa el catálogo de tres productos y las cinco rutas GET de la evidencia EV04. Corresponde a la solución utilizada para las pruebas registradas en las Figuras 11 a 17.

```
from fastapi import FastAPI, HTTPException

app = FastAPI(
    title="Mini-API de Catalogo",
    description="API REST para consultar productos de una tienda.",
    version="1.0.0"
)

productos = [
    {"id": 1, "nombre": "Teclado mecanico", "precio": 120000, "categoria": "Perifericos"},
    {"id": 2, "nombre": "Mouse gamer", "precio": 85000, "categoria": "Perifericos"},
    {"id": 3, "nombre": "Monitor 24", "precio": 650000, "categoria": "Pantallas"},
]

@app.get("/")
def inicio():
    return {"mensaje": "Bienvenido a la Mini-API de Catalogo"}

@app.get("/productos")
def listar_productos():
    return productos

@app.get("/productos/{producto_id}")
def obtener_producto(producto_id: int):
    for producto in productos:
        if producto["id"] == producto_id:
            return producto
    raise HTTPException(status_code=404, detail="Producto no encontrado")

@app.get("/categorias")
def listar_categorias():
    categorias = sorted({producto["categoria"] for producto in productos})
    return categorias

@app.get("/buscar")
def buscar_productos(nombre: str = ""):
    resultados = [
        producto
        for producto in productos
        if nombre.lower() in producto["nombre"].lower()
    ]
    return resultados
```

Conclusiones

Las evidencias de reflexión y comparación permitieron reconocer el papel de las API en la comunicación entre aplicaciones, distinguir las responsabilidades del frontend y el backend y relacionar HTTP, JSON, endpoints y códigos de estado con el consumo de servicios web.

El desarrollo del laboratorio confirmó que FastAPI permite crear servicios GET con una estructura sencilla y verificar sus operaciones desde Swagger UI. Se implementaron seis endpoints y se obtuvieron respuestas HTTP 200 en las pruebas previstas, lo que respalda el correcto funcionamiento de las rutas documentadas.

La prueba con un identificador no numérico permitió observar un error HTTP 422 sin programar una validación manual adicional. Esta funcionalidad, junto con la generación automática de la documentación, constituye una base práctica para abordar posteriormente la Mini-API de Catálogo con productos almacenados en memoria.

El reto de la Mini-API de Catálogo consolidó los conceptos de recursos REST y respuestas HTTP: se comprobó el listado de productos, la consulta individual, la deduplicación de categorías y la búsqueda por coincidencia de nombres. La respuesta 404 ante un identificador inexistente demostró un manejo explícito de errores. En conjunto, las cuatro evidencias proporcionan una base para futuras prácticas con modelos de datos, persistencia y seguridad.

Referencias bibliográficas

Arroyave, M. (2025). Formato guía de aprendizaje [Guía 1 de desarrollo de API REST con FastAPI]. Servicio Nacional de Aprendizaje (SENA).

FastAPI. (s. f.). Primeros pasos. https://fastapi.tiangolo.com/es/tutorial/first-steps/

MDN Web Docs. (s. f.). Generalidades del protocolo HTTP. https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview

Pallets Projects. (s. f.). Quickstart. Flask Documentation. https://flask.palletsprojects.com/en/stable/quickstart/

Uvicorn. (s. f.). Uvicorn: An ASGI web server, for Python. https://www.uvicorn.org/

Control de versiones

Tabla 4

Registro de control de versiones del documento

Versión	Fecha	Descripción de cambios	Integrantes
1.0	4/11/2026	Consolidación de EV01–EV04; incorporación de capturas y ajustes de formato, portada e índice.	Juan Esteban Lezcano Tejada Mariana Suárez
1.1	5/11/2026	Actualización de la portada y de la relación de integrantes del trabajo grupal.	Juan Esteban Lezcano Tejada Mariana Suárez Emanuel Villa

Nota. La versión 1.0 corresponde al documento consolidado de las cuatro evidencias de la Guía 1 de FastAPI. La versión 1.1 incorpora a Emanuel Villa como tercer integrante del equipo.